```
# Practical 4: Polynomial Regression

# 1. Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# 2. Load the dataset
diabetes = load_diabetes()

# Use one feature for easy visualization
X = diabetes.data[:, [2]]
y = diabetes.target

# 3. Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 4. Test different polynomial degrees
degrees = [1, 2, 3, 5, 10]

train_errors = []
test_errors = []
```

```
for degree in degrees:

    # Convert features into polynomial features
    poly = PolynomialFeatures(degree=degree)

    X_train_poly = poly.fit_transform(X_train)
    X_test_poly = poly.transform(X_test)

    # Train the model
    model = LinearRegression()
    model.fit(X_train_poly, y_train)

    # Predictions
    train_pred = model.predict(X_train_poly)
    test_pred = model.predict(X_test_poly)

    # Calculate errors
    train_mse = mean_squared_error(y_train, train_pred)
    test_mse = mean_squared_error(y_test, test_pred)

    train_errors.append(train_mse)
    test_errors.append(test_mse)

    print("\nPolynomial Degree:", degree)
    print("Training MSE:", train_mse)
    print("Testing MSE:", test_mse)
    print("R2 Score:", r2_score(y_test, test_pred))

# 5. Plot training and testing errors
plt.figure(figsize=(8, 5))
```

```
plt.plot(degrees, train_errors, marker='o',
        label='Training Error')

plt.plot(degrees, test_errors, marker='o',
        label='Testing Error')

plt.xlabel("Polynomial Degree")
plt.ylabel("Mean Squared Error")
plt.title("Bias-Variance Tradeoff")
plt.legend()
plt.grid(True)
plt.show()

# 6. Visualize polynomial regression curves
X_plot = np.linspace(X.min(), X.max(), 200).reshape(-1, 1)

plt.figure(figsize=(8, 5))
plt.scatter(X, y, color='blue', alpha=0.4, label='Actual Data')

for degree in [1, 2, 3, 5]:
    poly = PolynomialFeatures(degree=degree)

    X_poly = poly.fit_transform(X)
    X_plot_poly = poly.transform(X_plot)

    model = LinearRegression()
    model.fit(X_poly, y)

    y_plot = model.predict(X_plot_poly)

    plt.plot(X_plot, y_plot, label=f'Degree {degree}')
```

```
plt.xlabel("BMI Feature")
plt.ylabel("Disease Progression")
plt.title("Polynomial Regression with Different Degrees")
plt.legend()
plt.show()
```